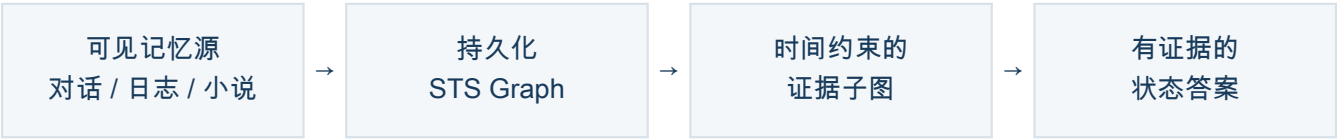


Scope-Time-State

Memory Graph

当前 Solution 整理 | AAI 方法说明

核心命题：长期记忆问答不是从历史中找一段相关文本，而是在正确的 Scope 和 Time 约束下，从持续演化的记忆流中读出仍然有效的 State。



适用输入：对话、群组消息、系统日志，以及 EPBench 一类按时间展开的合成长文本/小说。

2026-07 | EpisodicMemory

1. 论文要解决什么问题

长期记忆的难点不只是“相关内容在哪”，而是“在什么范围、什么时间语义下，哪个状态现在仍然有效”。

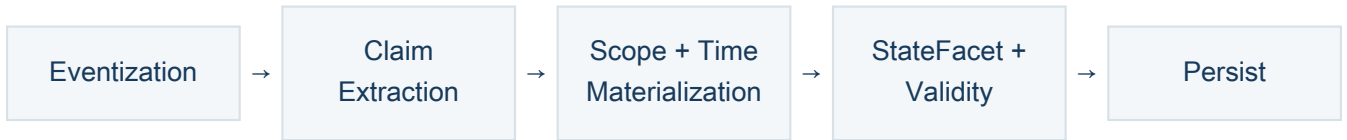
研究问题：给定一个按时间展开的长期记忆源和一个问题，构造目标范围内、由证据支撑的当前有效状态。

维度	问题
Scope	问题指向哪个项目、人物、主题、会话或情节对象？
Time	问题要求当前、发生、更新、计划、截止、开始还是完成语义？
Validity	一条 Claim 是否已经被纠正、覆盖、冲突或仅仅被复述？
State	要回答的是决策、风险、进度、下一步、偏好还是完成状态？

关键不是“最新事件”。计划不等于完成；最新消息也可能只是旧结论的复述。状态答案必须同时满足 Scope、Time 与 Validity。

2. 写入阶段：构建持久化 STS Graph

构图在问题出现之前完成。它只读取 benchmark-visible memory source，把事件流转换成带时间语义和状态关系的图。



- Eventization。对话中的 message/turn、日志记录或叙事片段成为 Event，保留顺序、文本、来源和时间锚点。
- Claim extraction。从 Event 中抽取 owner、decision、risk、status、deadline、next step、metric 等原子状态。
- Time materialization。为 Claim 写入 `time\_role`，例如 `occurred\_at`、`updated\_at`、`planned\_for`、`deadline\_at`、`started\_at`、`completed\_at`。
- State resolution。同一 Scope/subject/facet 的 Claims 被按时间和 `CORRECTS`、`SUPERSEDES`、`CONFLICTS\_WITH` 关系解析为 StateFacet。
- Persist。输出 nodes、edges、manifest 和 summary。问题、答案、选项、gold evidence 与 question type 不参与构图。

数据边界：EverMemBench 与 LoCoMo 读取 dialogue/conversation；GroupMemBench 读取群组消息；EPBench 读取合成长叙事文本。共同约束是：只读原始记忆源，不读 QA 或 gold。

3. Time 在图中做什么

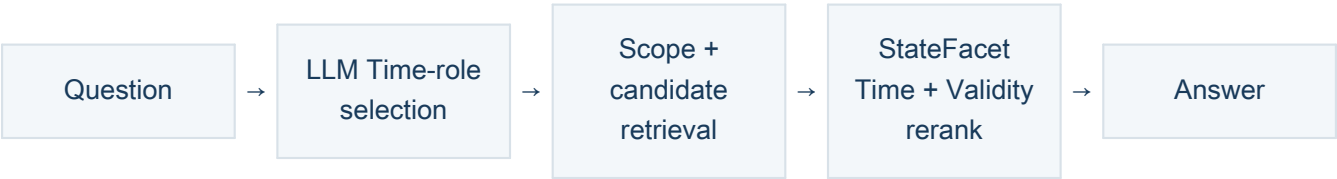
结构	Time 的作用
Event → Time OCCURRED_AT	记录事件何时发生，建立记忆流的时序锚点。
Claim → Time HAS_TIME + time_role	区分实际发生、更新、计划、截止、开始、完成等不同时间语义。
StateFacet → Time CURRENT_AFTER	记录一个状态从何时起有效，是“当前状态”而非“最新文本”的依据。
Claim → Claim CORRECTS / SUPERSEDES / CONFLICTS_WITH	表达状态更新、替代和冲突，决定哪些旧 Claim 不能直接当作当前结论。

例子：“6月1日计划使用 A”与“6月8日改为 B”会形成两个带不同 time role 的 Claim；后者通过 SUPERSEDES 影响前者，StateFacet 的当前值为 B，且 CURRENT_AFTER = 6月8日。

因此 Time 不是 Event 上的装饰日期。它同时定义 Claim 的语义、StateFacet 的生效时刻，以及后续查询应优先读哪一种状态证据。

4. 读出阶段：通用 Time-role selection

所有问题走同一条路径。Time selector 只读取问题文本，通过一次小型 LLM JSON 调用从固定 STS 时间本体中选择 1 至 3 个角色。



问题语义	selector 输出的例子
当前仍有效的状态	`CURRENT_AFTER`
计划或日程	`planned_for`
截止或到期	`deadline_at`
实际发生 / 更新 / 开始 / 完成	`occurred_at` / `updated_at` / `started_at` / `completed_at`

selector 的输入是 Question；它不读取 benchmark 名称、任务标签、选项、答案或 gold evidence。它不计算日期差，也不直接作答。关闭 selector 时，系统保守回退到 `CURRENT\_AFTER`。

StateFacet 重排：

score = retrieval score + validity score + time-role score。

其中 validity score 奖励当前有效、具备 CURRENT_AFTER、并通过 CORRECTS/SUPERSEDES 更新的状态；time-role score 奖励与 selector 输出角色匹配的 StateFacet，并降低角色不匹配的候选。

这不是按某个 benchmark 题面模板分支，而是 Scope-Time-State 查询读出中的共享语义约束。

5. 统一实现与论文主张

验证载体	当前 STS 接入
EverMemBench	topic-level STS graph；共享 Question-only selector；StateFacet 的 Time/Validity 重排。
LoCoMo-QA	sample-level role-aware v2 STS graph；使用同一 selector 与同一重排原则。
GroupMemBench	域级图上的 scope/time/state 状态读出，作为后续统一接入目标。
EPBench	合成长叙事事件图，用于跨段落/章节的 episodic 与时间状态验证。
LongMemEval-S	跨 session 的知识更新和时间推理外部验证。

AAAI 主线：当问题同时依赖范围判断、时间语义与状态有效性时，持久化 STS Graph 通过通用的 Time-role selection 和 Time/Validity-aware StateFacet rerank，组织出比最近事件或通用检索更可追溯的证据。

实现锚点：共享模块 `pipeline/external/time\_role\_selection.py`；EverMemBench `Baseline/ours\_scope\_time\_state/staged.py` 与 `qa\_eval\_runner.py`；LoCoMo `Baseline/ours\_scope\_time\_state/graph\_query\_runner.py`。